26-1 Intelligent Robotics

2129032 장한나

GestureBot

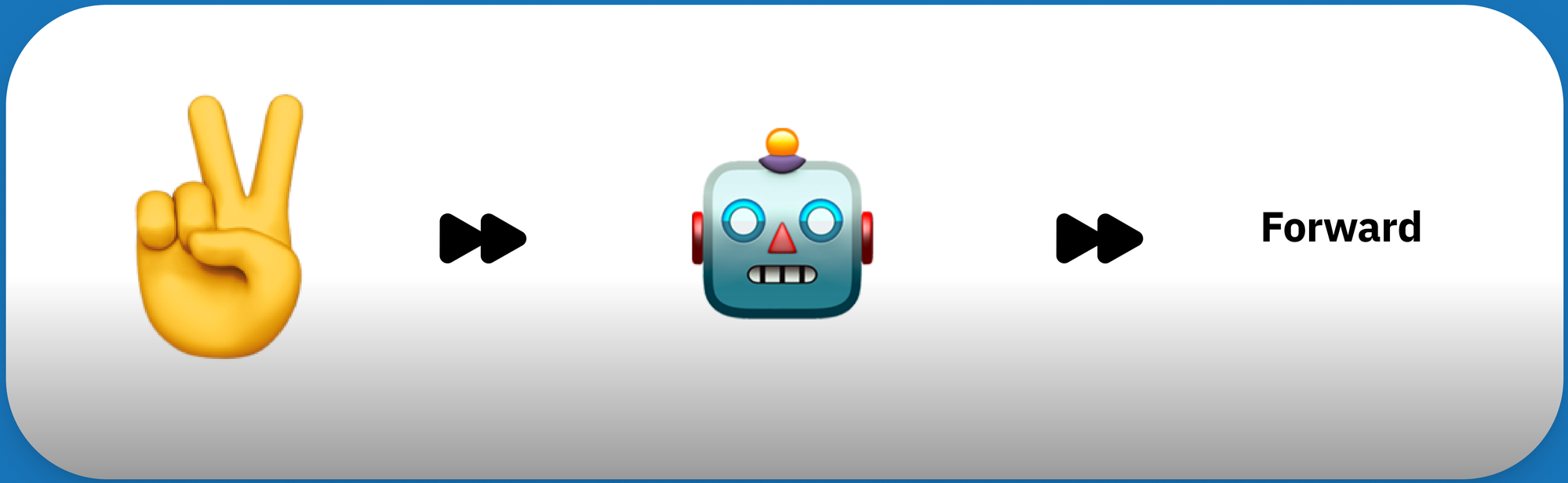
: 손동작 기반 모바일 로봇 제어 시스템

목차

1. GestureBot
2. System Architecture
3. MediaPipe
4. 구현상세
    - 기본 동작
    - Nav2 기반 동작
5. Demo
6. 의의 및 향후 발전 방향

GestureBot

- **사람의 손동작**을 인식하여 이를 기반으로 이동하는 로봇 시스템



GestureBot: 개발환경

- **Framework**: Ros2 Humble
- **Simulator**: Gazebo Fortress
- **Robot**: Turtlebot4
- **Image Processing**: OpenCV
- **Gesture Recognition**: MediaPipe (Tasks API)
- **Navigation**: Nav2
- **Visualization**: Rviz

Gesturebot: 동작 개요

1. 카메라를 통해 사람의 손동작 이미지 정보 수신
2. 손동작 인식 및 분류
3. 손동작에 지정된 행동을 수행

손동작	로봇동작
👌 OK 손 모양	→ 정지
✌️ V자 손 모양	→ 직진
🖐️ 손바닥 펼치기	→ 후진
👉 오른쪽으로 가리키기	→ 오른쪽으로 회전
👈 왼쪽으로 가리키기	→ 왼쪽으로 회전
손을 위로 들기	→ 속도 증가
손을 아래로 들기	→ 속도 감소
손가락으로 3을 표시	→ 목표 지점까지 자율 주행

Gesturebot: 동작 개요

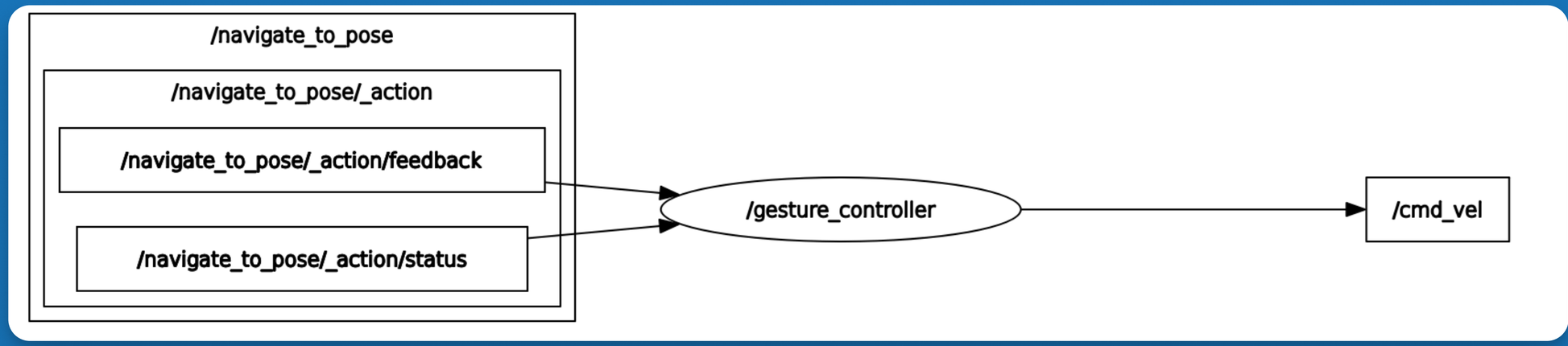
1. 카메라를 통해 사람의 손동작 이미지 정보 수신
2. 손동작 인식 및 분류
3. 손동작에 지정된 행동을 수행

손동작	로봇동작
👌 OK 손 모양	→ 정지
✌️ V자 손 모양	→ 직진
🖐️ 손바닥 펼치기	→ 후진
👉 오른쪽으로 가리키기	→ 오른쪽으로 회전
👈 왼쪽으로 가리키기	→ 왼쪽으로 회전
손을 위로 들기	→ 속도 증가
손을 아래로 들기	→ 속도 감소
손가락으로 3을 표시	→ 목표 지점까지 자율 주행

→ Nav2 기반

System Architecture

- rqt 그래프
- **gesture_controller node**를 통해 제어



System Architecture

- 코드 구조

```
motionbot_ws/
├── src/
│   └── gesture_control/
│       ├── gesture_control/
│       │   ├── __init__.py
│       │   ├── gesture_classifier.py   # 제스처 분류 로직
│       │   ├── gesture_controller.py   # ROS2 메인 노드
│       │   ├── gesture_to_cmd.py       # 제스처 → cmd_vel 변환
│       │   └── nav_goal_publisher.py   # Nav2 목표 지점 전송
│       ├── launch/
│       │   └── gesture_control.launch.py
│       ├── package.xml
│       ├── setup.cfg
│       └── setup.py
├── hand_landmarker.task
├── build/
├── install/
└── log/
```

MediaPipe

- 오픈소스 ML 프레임워크, 실시간 미디어 처리에 특화

- MediaPipe Hand Landmarker 모델
  - 웹캠 이미지에서 손의 21개 관절 좌표 실시간 추출
  - Tasks API 방식 사용

- Tasks API
  - 기존 Legacy API보다 안정적
  - numpy 버전 충돌 문제 없음

구현상세: 기본 동작

- 손동작에 따라 정지, 직진, 후진, 회전, 증감속 동작

/gesture_contoller

/cmd_vel

- 웹캠을 통해 손동작 이미지 인식

- 이미지 전처리
- gesture_classifier.py: 손동작 분류
- gesture_to_cmd.py: Twist 메시지 생성

- Twist 메시지 기반 로봇 제어

gesture_controller.py

- ROS2 /gestsure_controller node 생성 및 관리
- 웹캠 이미지 수신 및 전처리 (OpenCV)
- 손 랜드마크 추출 (MediaPipe)
- 제스처 분류 및 명령 변환 (gesture_classifier, gesture_to_cmd)
- /cmd_vel 퍼블리시

gesture_controller.py

- ROS2 /gestsure_controller 노드 생성 및 관리
- 웹캠 영상 수신 (OpenCV), 손 랜드마크 추출 (MediaPipe)

```python
class GestureController(Node):
    def __init__(self):
        super().__init__('gesture_controller')

        self.speed = 1.0
        self.angular = 0.3
        self.last_direction = 'stop'
        self.nav_mode = False
        self.latest_frame = None  # 메인 스레드에서 표시할 프레임

        self.publisher = self.create_publisher(Twist, '/cmd_vel', 10)
        self.nav_goal_publisher = NavGoalPublisher()

        self.cap = cv2.VideoCapture(0)

        base_options = python.BaseOptions(model_asset_path=MODEL_PATH)
        options = vision.HandLandmarkerOptions(base_options=base_options, num_hands=1)
        self.detector = vision.HandLandmarker.create_from_options(options)

        self.timer = self.create_timer(0.05, self.process_frame)
```

```python
def process_frame(self):
        ret, frame = self.cap.read()
        if not ret:
            return

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
        result = self.detector.detect(mp_image)
        msg = Twist()
        should_publish = True

        if result.hand_landmarks:
            for hand_landmark in result.hand_landmarks:
                for lm in hand_landmark:
                    h, w, _ = frame.shape
                    cx, cy = int(lm.x * w), int(lm.y * h)
                    cv2.circle(frame, (cx, cy), 3, (0, 255, 0), -1)
```

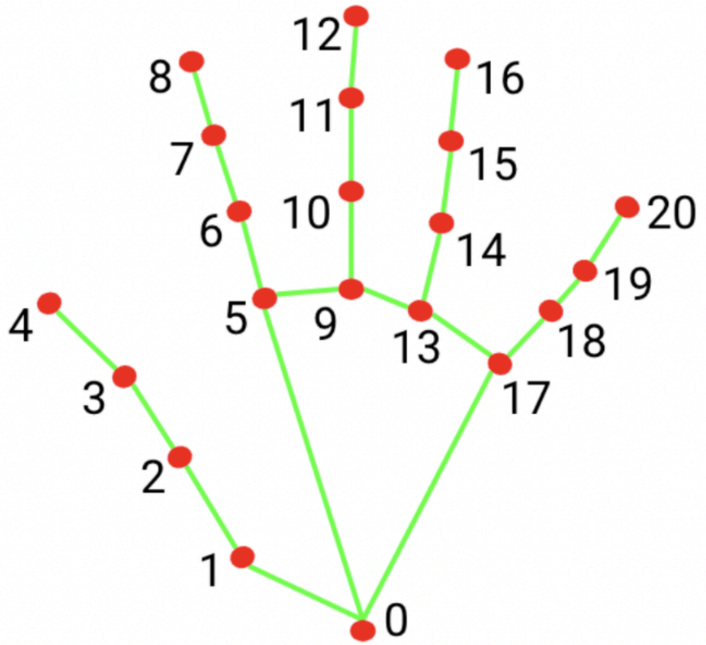
gesture_controller.py

- 제스처 분류 및 명령 변환 (gesture_classifier, gesture_to_cmd)
- /cmd_vel 퍼블리시

```
            gesture = recognize_gesture(result.hand_landmarks[0])
            cv2.putText(frame, gesture, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

              if gesture == 'goal' and not self.nav_mode:
                self.nav_mode = True
                self.nav_goal_publisher.send_goal(done_callback=self.on_nav_done)
                msg = gesture_to_twist('stop', self.speed, self.angular)

              elif self.nav_mode:
                if gesture == 'stop':
                    self.nav_goal_publisher.cancel_goal()
                    self.nav_mode = False
                should_publish = False

              elif gesture in ('speed_up', 'speed_down'):
                self.speed = update_speed(gesture, self.speed)
                gesture = self.last_direction
                msg = gesture_to_twist(gesture, self.speed, self.angular)

              else:
                self.last_direction = gesture
                msg = gesture_to_twist(gesture, self.speed, self.angular)

          else:
            msg = gesture_to_twist('stop', self.speed, self.angular)
```

gesture_classifier.py

- mediapipe가 인식한 hand landmark 기반 손동작 분류



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

gesture_classifier.py

- 예시)
  - 손목(0)의 x 좌표 < 검지 끝(8)의 x 좌표 → 우측으로 회전
  - 손목(0)의 x 좌표 > 검지 끝(8)의 x 좌표 → 좌측으로 회전

```
# 오른쪽을 가리킴: 검지 손가락이 오른쪽을 가리킴-> 오른쪽으로 회전
# 왼쪽을 가리킴: 검지 손가락이 왼쪽을 가리킴 -> 왼쪽으로 회전
if index_extended and not middle_extended and not ring_extended and not pinky_extended:
    if lm[0].x < lm[8].x:
        return 'right'
    else:
        return 'left'
```

gesture_to_cmd.py

- 인식한 손동작을 기반으로 Twist 메시지 생성
- Twist의 linear.x와 angular.z를 조절

```python
def gesture_to_twist(gesture, speed=0.3, angular=0.3):
    msg = Twist()

    if gesture=='forward':
        msg.linear.x = speed
    elif gesture=='backward':
        msg.linear.x = -speed
    elif gesture =='left':
        # msg.linear.x = speed
        msg.angular.z = angular
    elif gesture == 'right':
        # msg.linear.x = speed
        msg.angular.z = -angular

    return msg
```

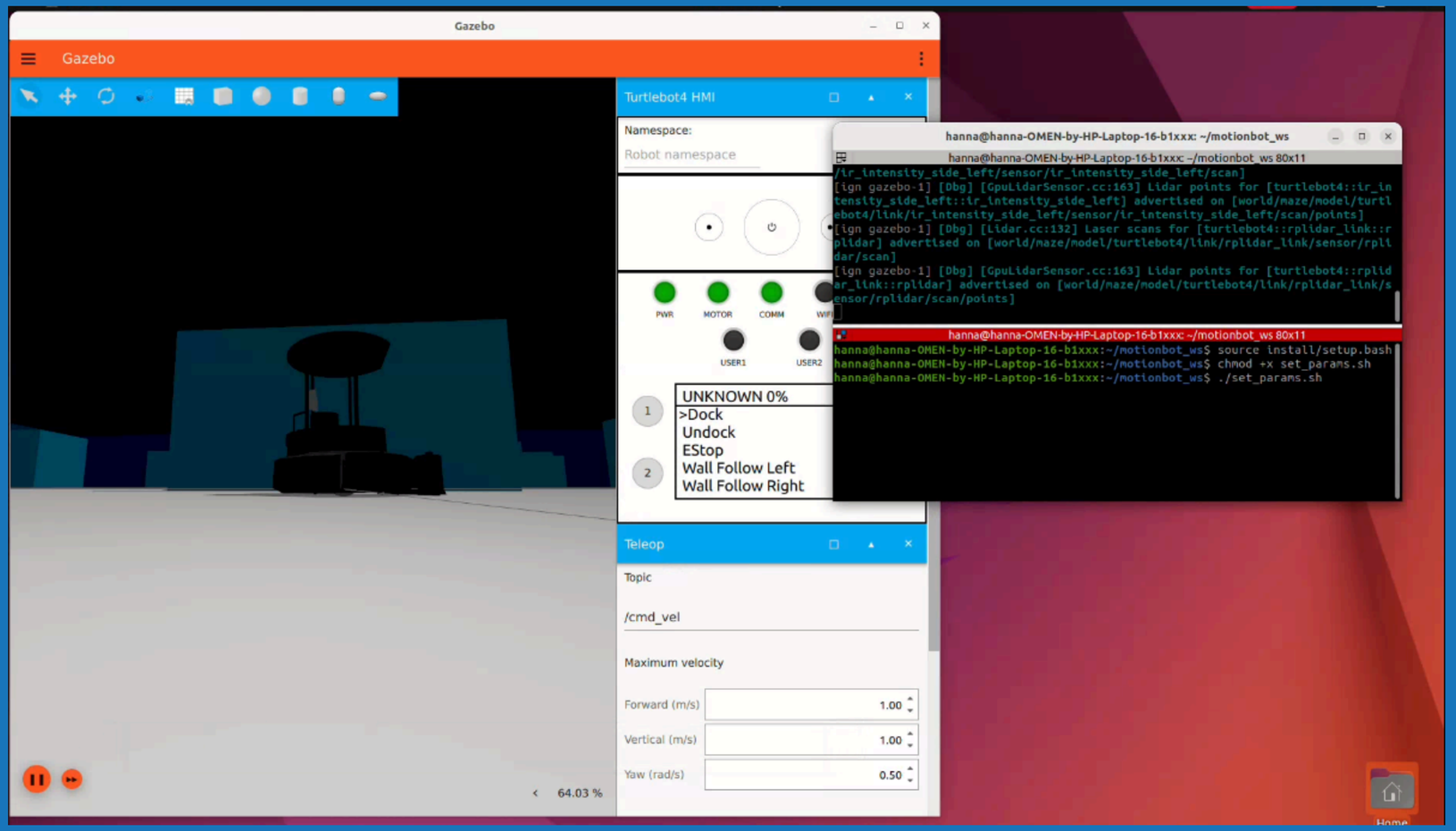
```python
def update_speed(gesture, speed):
    if gesture == 'speed_up':
        speed = min(speed+0.1, 5.0) # 최대 5.0
    elif gesture == 'speed_down':
        speed = max(speed-0.1, 0.1) # 최소 0.1
    return speed
```

Issues & Solutions

- Turtlbot4 자체 후진 속도 제약
  - 문제: Create3 기반 로봇의 후진 속도 제한 ⚠️
  - 해결: /motion_control 노드의 safety_override 파라미터를 full로 설정 ✅

- 손동작 인식 오류
  - 문제: 유사한 손동작 간 혼동 발생 ⚠️
  - 해결: 제스처 변경 ✅
    - 정지: 주먹 쥐기(기존) → OK자 손모양 (현재)
    - 직진: 앞을 가리키기(기존) → V자 손모양 (현재)

Demo: 기본동작



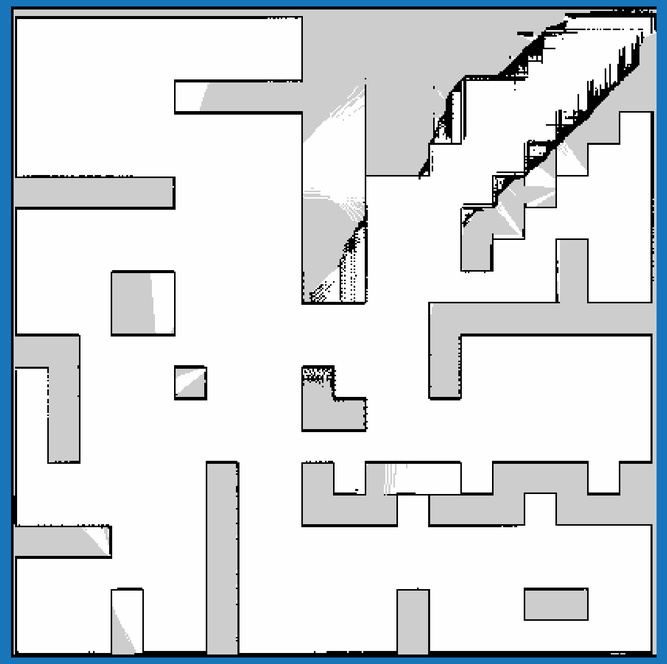
Demo 영상

구현상세: Nav2 기반 동작

- Nav2 기반 자율주행
- 특정 손동작을 하면 미리 정해둔 목표 지점으로 이동

Nav2 사용 기본 설정

1. Nav2 패키지 설치
2. **SLAM**으로 map의 지도 생성 및 저장
3. Localization&Nav2 실행



nav_goal_publisher.py

- gesture_controller 내부에서 Nav2와의 통신 담당
- NavigateToPose goal 전송 및 결과 수신
- 그 외 goal 관련 정보 및 기능 관리

nav_goal_publisher.py

- gesture_controller 내부에서 Nav2와의 통신 담당
- NavigateToPose goal 전송 및 결과 수신

```python
def send_goal(self, done_callback=None):
    goal_msg = NavigateToPose.Goal()
    goal_msg.pose.header.frame_id = 'map'
    goal_msg.pose.header.stamp = self.get_clock().now().to_msg()
    goal_msg.pose.pose.position.x = self.goal['x']
    goal_msg.pose.pose.position.y = self.goal['y']
    goal_msg.pose.pose.orientation.w = 1.0

        self._done_callback = done_callback
    # self._is_active = True
    future = self._action_client.send_goal_async(goal_msg)
    future.add_done_callback(self.goal_response_callback)
    self.get_logger().info('목표 지점 전송!')
```

```python
def goal_result_callback(self, future):
        try:
            result = future.result()
            from action_msgs.msg import GoalStatus
            if result.status == GoalStatus.STATUS_SUCCEEDED:
                self.get_logger().info('목표 지점 도착!')
                if self._done_callback:
                    self._done_callback()
            elif result.status == GoalStatus.STATUS_ABORTED:
                self.get_logger().warn('내비게이션 실패 (aborted)')
                if self._done_callback:
                    self._done_callback()
            elif result.status == GoalStatus.STATUS_CANCELED:
                self.get_logger().info('내비게이션 취소됨')
                # nav_mode는 cancel_goal()에서 이미 False로 설정됨
            else:
                self.get_logger().warn(f'예상치 못한 상태: {result.status}')
        except Exception as e:
            self.get_logger().error(f'Result callback error: {e}')
```

nav_goal_publisher.py

- 그 외 goal 관련 정보 및 기능 관리
  - 목표 지점 관리
  - 목표 취소 기능

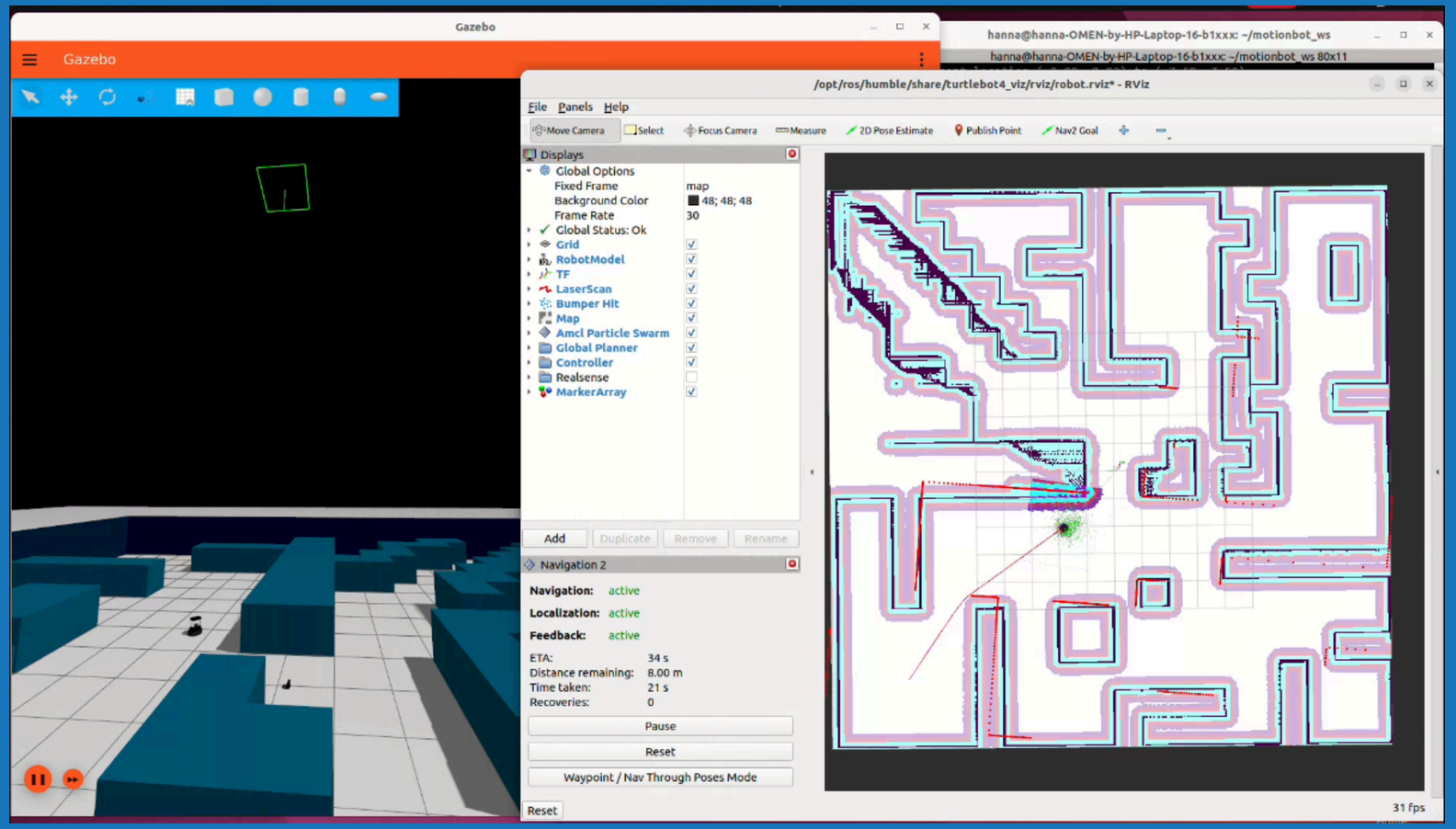
```
self.goal = {'x': -7.5, 'y': 7.5}
def cancel_goal(self):
        if self._goal_handle is not None:
            self._goal_handle.cancel_goal_async()
            self.get_logger().info('목표 취소!')
```

Issues & Solutions

- Nav2 도착 인식 오류
  - 문제: 목표 지점 도달 후에도 도착으로 인식하지 못함 ⚠️
  - 해결:
    - /controller_server의 **goal_checker tolerance** 완화 ✅
      (xy_goal_tolerance: 0.5m, yaw_goal_tolerance: 3.14rad)
    - localization & Nav2 실행 시 **use_sim_time=true** 설정 ✅
      (시뮬레이션 시간과 ROS2 시간 불일치 해결)

Demo: Nav2 기반 동작

- Stop 동작으로 자율주행 → 수동모드 전환 가능



Demo 영상

✶ Demo: 전체 동작 흐름

의의 및 향후 발전 방향

- 의의
  - 별도의 입력 장치 없이 손동작만을 이용한 로봇 제어 시스템 구축
  - 손동작 제어(수동)와 Nav2 자율주행을 하나의 시스템에서 통합 구현

- 향후 발전 방향
  - 손동작 인식률 개선
  - YOLO 등 Object Detection 연동으로 물체 기반 자율 회피/접근 기능 추가

참고자료

TurtleBot4 공식 매뉴얼
- https://turtlebot.github.io/turtlebot4-user-manual/software/turtlebot4_simulator.html

OpenCV GitHub
- 이미지 처리 https://github.com/ros-perception/vision_opencv

MediaPipe GitHub&공식 매뉴얼
- https://github.com/google-ai-edge/mediapipe
- https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker?hl=ko

MediaPipe+Turtlebot3
- https://github.com/Srikiranrao/ROS-mediapipe-turtlebot3

✶ 감사합니다

GitHub